

德州学院

毕业设计



题 目 基于微信小程序的安琦购平台的设计
与实现

指导教师 董光智、郭超

学生姓名 赵汝庆

学 号 202200705166

专 业 数据科学与大数据技术

教学单位 数学与大数据学院 （盖章）

二〇二六年五月五日

德州学院

毕业论文(设计)诚信声明

郑重声明：本毕业论文(设计)是我个人在指导教师指导下，进行研究工作所取得的成果，成果不存在知识产权争议。除文中已经注明引用的内容外，本文不含任何其他个人或集体已经发表或撰写过的作品成果，也不包含已获得德州学院或者其他单位的学位或证书所使用过的材料。对本文的研究做出重要贡献的个人和集体在文中均作了明确的说明并表示了谢意。本人完全理解有关学术诚信和道德规范，知晓可能引发的法律后果，愿意承担由此导致的法律责任。

毕业论文(设计)作者签名：赵汝东

2026 年 4 月 30 日

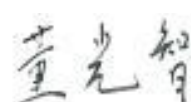
学位论文版权使用授权书

本学位论文作者完全了解德州学院有关保留、使用学位论文的规定，有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许论文被查阅和借阅。本人授权德州学院可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

(保密的学位论文在解密后适用本授权书)

学位论文作者签名：

签字日期：2026年5月6日

指导教师签名：

签字日期：2026年5月6日

目 录

摘要及关键词.....	1
1 引言	1
1.1 选题目的和意义	1
1.2 国内外的研究现状	2
2 系统关键技术与架构原理	3
2.1 基于 Spring Cloud Alibaba 的微服务架构原理	3
2.2 跨平台前端技术与高性能搜索原理	4
3 系统需求分析	6
3.1 功能性需求分析	6
3.2 非功能性需求分析	7
4 系统总体架构与数据库设计	8
4.1 系统总体逻辑架构设计	8
4.2 关系型数据库逻辑设计	11
4.3 搜索引擎与缓存架构设计	14
5 核心功能模块的详细设计与实现	15
5.1 用户登录模块设计与实现	15
5.2 前端交互与状态管理实现	16
5.3 商品检索与渲染模块实现	18
5.4 订单交易与支付模块实现	20
5.5 物流跟踪与消息推送实现	22
6 系统测试与性能优化	25
6.1 系统功能测试	25
6.2 系统性能优化策略	27
7 总结	27
参考文献	29
谢 辞	31

Contents

Abstract and Keywords	1
1 Introduction	1
1.1 Purpose and Significance	1
1.2 Research Status at Home and Abroad	2
2 System Key Technologies and Architecture	3
2.1 Spring Cloud Alibaba Microservice Architecture	3
2.2 Cross-platform Front-end and High-performance Search	4
3 System Requirement Analysis	6
3.1 Functional Requirements	6
3.2 Non-functional Requirements	7
4 System Architecture and Database Design	8
4.1 Overall Logical Architecture	8
4.2 Relational Database Logical Design	11
4.3 Search Engine and Cache Architecture	14
5 Design and Implementation of Core Modules	15
5.1 Design and Implementation of User Login Module	15
5.2 Front-end Interaction and State Management	16
5.3 Commodity Retrieval and Display	18
5.4 Order Transaction and Payment	20
5.5 Logistics Tracking and Message Push	22
6 System Testing and Performance Optimization	25
6.1 Functional Testing	25
6.2 Performance Optimization Strategies	27
7 Summary	27
References	29
Acknowledgements	31

基于微信小程序的安琦购平台的设计与实现

赵汝庆

(德州学院数学与大数据学院, 山东德州 253023)

摘 要: 在如今大型企业级电商平台的开发工作中, 微服务架构已然成为一项至关重要的系统设计准则, 且在提升系统高并发处理效率、增强业务扩展灵活性等多个层面, 该架构都有着诸多可落地的实际价值; 一方面, 结合“安琦购”微信小程序的完整开发案例进行剖析, 可让读者清晰了解到 Spring Cloud Alibaba 与 uni-app 搭配的技术方案, 在全链路电商系统中的运行逻辑, 另一方面, 顺着整套开发流程, 还能更透彻地讲解如何运用架构核心思想, 妥善处理跨平台适配、商品高性能检索、分布式业务治理等项目开发中实际遇到的工程难题。

关键词: 微服务; 微信小程序; Spring Cloud Alibaba; Elasticsearch

1 引言

在本章节内容中, 深度观察整个电商行业的背景情况及当前研究动态后, 明确了传统单体式架构在运行过程中暴露的各类痛点, 进而进一步确定了研究的具体方向。

1.1 选题目的和意义

在电子商务领域实现了快速发展之后, 广大群众原本的消费习惯已经被彻底改变了, 这也给电商系统的规划与开发工作带来了不少实际困难, 其中最核心的阻碍集中在系统架构规划与前端交互设计这两个关键环节; 电商平台发展早期, 开发者大多会将单体架构应用到系统开发与部署中, 会把用户管理、商品展示以及订单处理等所有业务逻辑统一整合到一个系统内部处理, 在平台刚起步、业务规模较小的时候, 这种方式有着较好的维护便利性, 具体操作流程也比较简单直接; 但每当“618”、“双十一”这类高流量、高并发的促销节点到来时, 这种架构就容易出现各类问题, 数据库读写卡顿、服务响应迟缓都是常见情况, 严重时还可能引发服务雪崩, 此外, 由于其内部各模块关联度过高, 系统进行迭代升级时, 就必须重新部署全部内容, 这不仅会耗费大量时间与人力, 还严重限制了系统的扩展能力与维护效率, 给后续优化工作带来了诸多麻烦^[1]。

针对传统电商系统在架构和前端开发中暴露出的上述问题, 这次课题的研究目的, 就是要将“安琦购”这个基于微信小程序的综合性电商平台, 给设计并完整实现出来, 通过引入 Spring Cloud Alibaba 微服务架构和 uni-app 跨平台前端技术, 打造出一套高性能、易扩展、多端适配的电商系统。

选题的意义主要能体现在以下两个方面:

工程应用价值, 这次开发的安琦购平台, 把 Elasticsearch 全文搜索引擎和 Redis 分布式缓存技术给深度整合了进来, 能有效解决掉海量商品数据检索慢、热点数据访问响应迟这类电

商行业里很常见的痛点，这套系统的设计和实现方案，能给中小型企业打造电商平台提供可复用的参考，让中小企业在做电商数字化转型的时候，能降低技术门槛和研发成本，有很强的实际落地价值。

学术研究意义，在本次平台开发与研究的过程中，实际摸索出了 Vue3 组合式 API 结合 TypeScript，在大型前端项目中的工程化使用方式，同时也测试并验证了 JWT 无状态认证在微服务网关中的安全实现方式。这些实际的实践成果，可以给全栈开发模式下的前后端分离架构，提供一些可参考的具体实现思路，对同类技术的研究和实际应用来说，也有一定的借鉴价值。

1.2 国内外的研究现状

从电商系统的整个发展历程来看，架构设计已经走过了从单体架构、面向服务架构也就是 SOA，再到微服务架构的逐步变革过程，国内外的技术人员和研发团队，都围绕电商平台的性能优化、架构升级展开了大量的研究和实践，也形成了各自的技术特点，同时整个行业也呈现出了一些统一的发展趋势。

国外研究现状：微服务架构的实践在国外起步要更早一些，像 Netflix、Amazon 这类科技巨头，是最早把微服务架构应用到电商平台开发里的企业，他们根据自身的业务需求搭建了完善的微服务体系，也推动了 Spring Cloud Netflix 生态的成熟，Eureka、Hystrix 这些组件也一度成了微服务开发的常用工具，可随着技术的发展，Spring Cloud Netflix 里的部分组件慢慢进入了维护模式，不再做新的功能更新，轻量级、云原生的微服务解决方案，慢慢成了国外技术社区的研究重点，而在前端开发领域，近几年“Headless Commerce 也就是无头电商”的概念兴起并快速发展，这个概念的核心就是把前端展示层和后端业务逻辑层彻底分离开，通过 API 驱动实现前端界面的灵活开发，能最大程度地优化用户体验，也成了国外电商前端开发的重要发展方向。

国内研究现状：对电商技术开发者来说，“双 11” 的流量峰值倒逼国内摸索出贴合本土需求的高并发架构体系，摆脱了对国外模式的依赖。

微服务开发中，以往的 Eureka 生态对中文开发者不够友好，而阿里开源的 Spring Cloud Alibaba (Nacos、Sentinel、Seata 等)，凭借中文适配性和出色的高并发表现，逐渐成为国内微服务开发的首选。

微信小程序爆发后，重复开发多终端的痛点凸显，uni-app、Taro 等跨端框架凭借“一次开发，多端部署”的优势兴起，成为行业通用标准，也是移动端开发者的常用工具^[2]。

2 系统关键技术与架构原理

本章拆解了安琦购平台核心技术栈的运行逻辑与应用方法，为系统开发奠定了基础。后端梳理 Spring Cloud Alibaba 核心组件功能与协同逻辑；前端及检索研究中，分析了相关技术原理，依托 Elasticsearch 实现全文检索，解决了多端适配与检索低效问题。

2.1 基于 Spring Cloud Alibaba 的微服务架构原理

单体应用会把所有业务逻辑都集中在同一个进程里，业务一扩展就容易出现代码耦合、部署繁琐这类问题，而 Spring Cloud Alibaba 作为国内现在主流的微服务框架，能很好的实现业务模块的解耦和高效管理，这套系统就是靠着这个框架来搭建微服务架构的，会把电商业务拆分成用户、商品、订单等独立的微服务模块，各模块通过标准化的轻量级接口完成通信，实现独立的开发、部署与扩展。

2.1.1 Nacos 服务注册发现与动态配置管理

当系统实际运行到分布式环境中时，各服务实例对应的网络地址，会随着具体的部署动作发生诸多动态变化，若直接采用硬编码的方式固定地址，后续维护系统所需耗费的成本便会大幅度增加；为解决这一核心难题，本系统专门引入了 Nacos 组件，一方面让其充当服务注册中心，另一方面也使其充分发挥配置中心的效用，如图 1 所示，微服务架构下的 Nacos，主要承担着服务注册与发现、动态配置管理这两大关键职能。

在服务注册与发现的这块，各微服务启动的时候，会通过 bootstrap.yml 配置，把自身的服务名、IP、端口这些元数据都注册到 Nacos Server 里，服务消费者要调用服务的时候，不用指定具体的 IP，只需要向 Nacos 拉取对应服务的健康实例列表，再结合负载均衡算法选一个实例调用就可以了，同时 Nacos 客户端会保持定时的心跳检测，要是某一个实例宕机了，Nacos 会自动把它从服务列表里剔除掉，能很好的保障服务调用的可靠性。

在动态配置管理的这块，系统把数据库连接、Redis 地址、日志级别这些配置信息，都统一托管到了 Nacos 里，当需要修改配置的时候，只需要在 Nacos 控制台操作就可以，配置变更的事件会被各微服务实时监听，实现配置的动态刷新，不用重启服务，把运维的效率提了一大截。

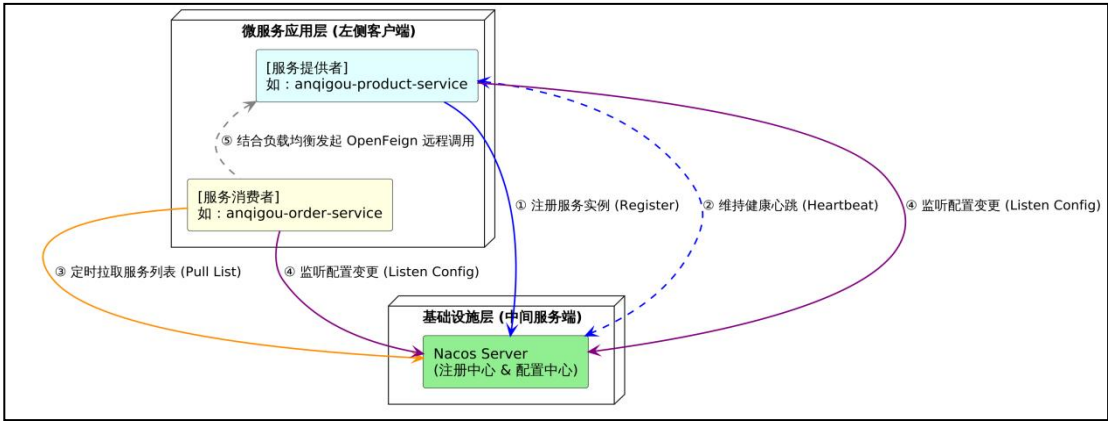


图 1 Nacos 服务注册与配置动态更新交互流向图

2.1.2 Spring Cloud Gateway 网关路由与过滤器原理

这套系统的微服务架构里，Spring Cloud Gateway 就是统一的流量入口。它是基于非阻塞的响应式编程模型做的，处理高并发请求特别给力，平时主要扛下这几个核心工作——请求路由分发、全局身份认证，还有跨域处理。

一是动态路由匹配机制，网关通过 Path Route Predicate Factory 实现动态的路由匹配，把路由信息都定义在配置文件里，包含 ID、目标 URI、断言和过滤器四个部分，当客户端发起请求的时候，网关会根据 URL 路径和断言规则做匹配，匹配成功后就把请求转发到对应的微服务里；

二是过滤器链处理流程，自定义了全局认证过滤器 JwtAuthFilter，会先校验请求路径是不是在白名单里，公开接口就直接放行，非白名单的请求则提取请求头里的 Token，校验它的签名和有效期，无效的话就直接返回 401 未授权的提示，把认证逻辑从业务微服务里剥离了出来，降低了系统的耦合度^[3]。

2.2 跨平台前端技术与高性能搜索原理

这套系统需要同时支持微信小程序和 H5 终端，还要实现海量商品的毫秒级检索，所以前端就采用 uni-app 结合 Vue3 实现跨平台开发，后端引入 Elasticsearch 全文搜索引擎，针对性的解决多端适配和高性能检索的业务痛点。

2.2.1 uni-app 跨端编译机制与 Vue3 响应式原理

uni-app 是依托 Vue.js 构建的跨平台开发框架，它最突出的优势就是能够实现一套代码多端发布，开发者只需要维护一份代码，就能编译出微信小程序、H5 等不同平台的应用。该框架在底层依靠条件编译来适配各平台的特有业务逻辑，还借助 AST 语法树转换，把 Vue 的模板和样式代码转换成对应平台的原生语法，很大程度上减少了多端开发和适配的重复工作。

本系统前端选用了 Vue3 组合式 API, 通过 ES6 的 Proxy 实现数据的响应式效果. 和 Vue2 采用的 Object.defineProperty 相比, Proxy 能够直接监听数组的下标和长度变化, 工作原理如图 2 所示, 很好地弥补了旧版本在数组监听上的不足, 同时也降低了大型数据对象初始化时的性能消耗, 让移动端页面的渲染更流畅, 用户在操作使用时的体验也有了明显提升^[4].

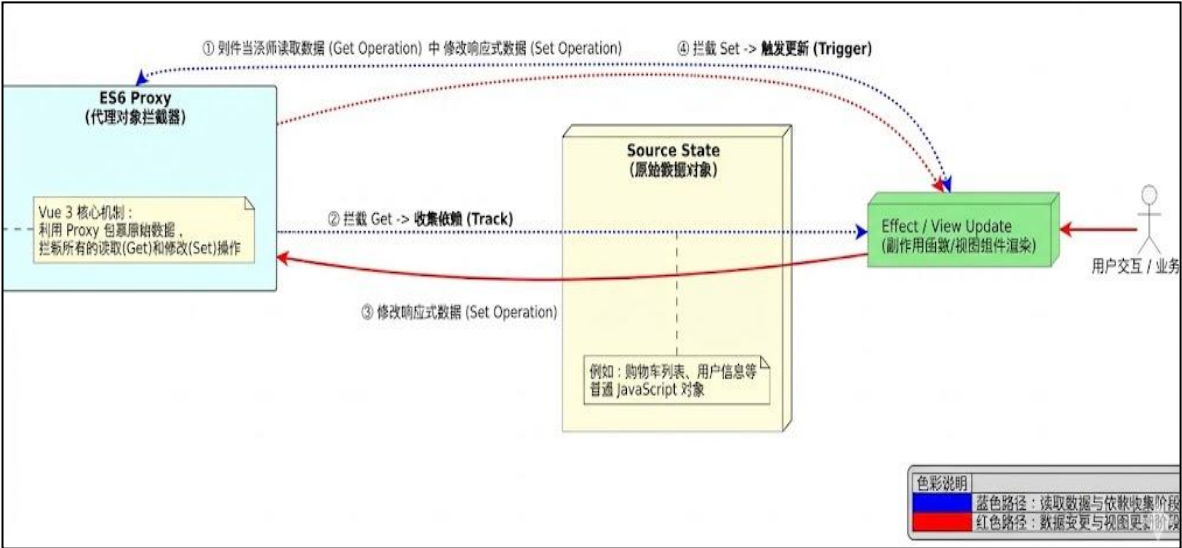


图 2 Vue3 响应式系统工作原理图

2.2.2 Elasticsearch 倒排索引与全文检索机制

为了解决 MySQL 在海量数据模糊查询下出现的性能瓶颈, 开发的系统里已经引入了 Elasticsearch7.17 作为核心的搜索引擎, 它主要是用来做商品搜索和分类筛选这两项工作的; 系统里的索引映射设计, 在这项设计工作中, 采用了“Text+Keyword”的混合策略, 像商品名称、品牌这些字段都设成了 Text 类型, 还配置了 ik_max_word 分析器来实现中文分词, 以此提升搜索的召回率, 而商品分类这类字段则设为 Keyword 类型, 不做任何分词处理, 以此保障分类筛选的精确性, 如图 3 所示, 这种设计刚好就适配了平台“模糊搜+精准筛”的商品检索需求.

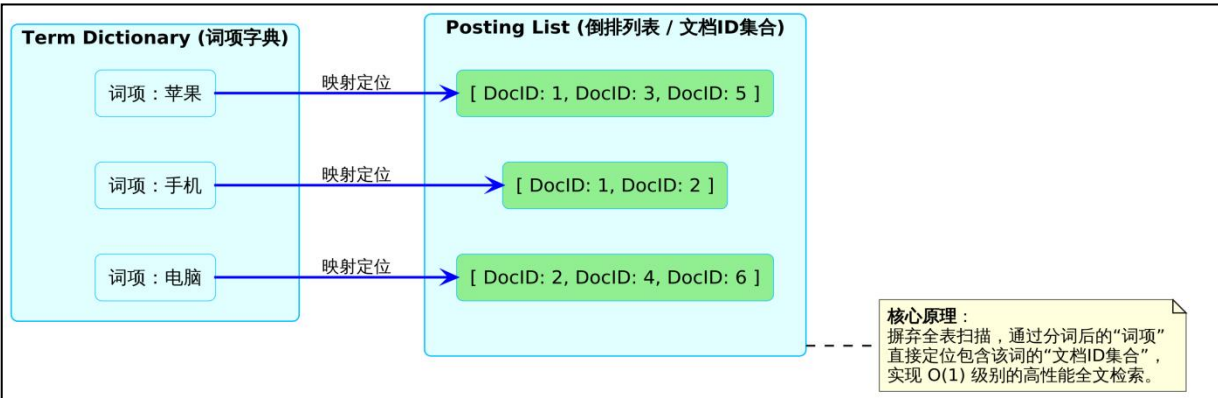


图 3 倒排索引结构示意图

3 系统需求分析

本章把需求分析给看作系统设计的前置关键，在结合了用户想法并看到了行业痛点以后，把安琦购平台的需求给理顺了：一方面能把业务目标与定位给找准，另一方面把功能需求给拆开并划了模块边界，此外还把非功能需求给讲透了；这也就为后续设计划到了基准。

3.1 功能性需求分析

看到这些年电商行业发展得挺快，用户对线上购物够不够快、好不好用也有了要求，可老平台把多端适配给搞复杂了，流程也杂，物流更不透明；这次把安琦购平台给放到了微信小程序里，能把购物环节给优化了，会用到 Spring、Vue 与数据库技术，把展示、下单、物流跟踪都给集成到这个轻量系统里，具体的用例情况如图 4 所示。

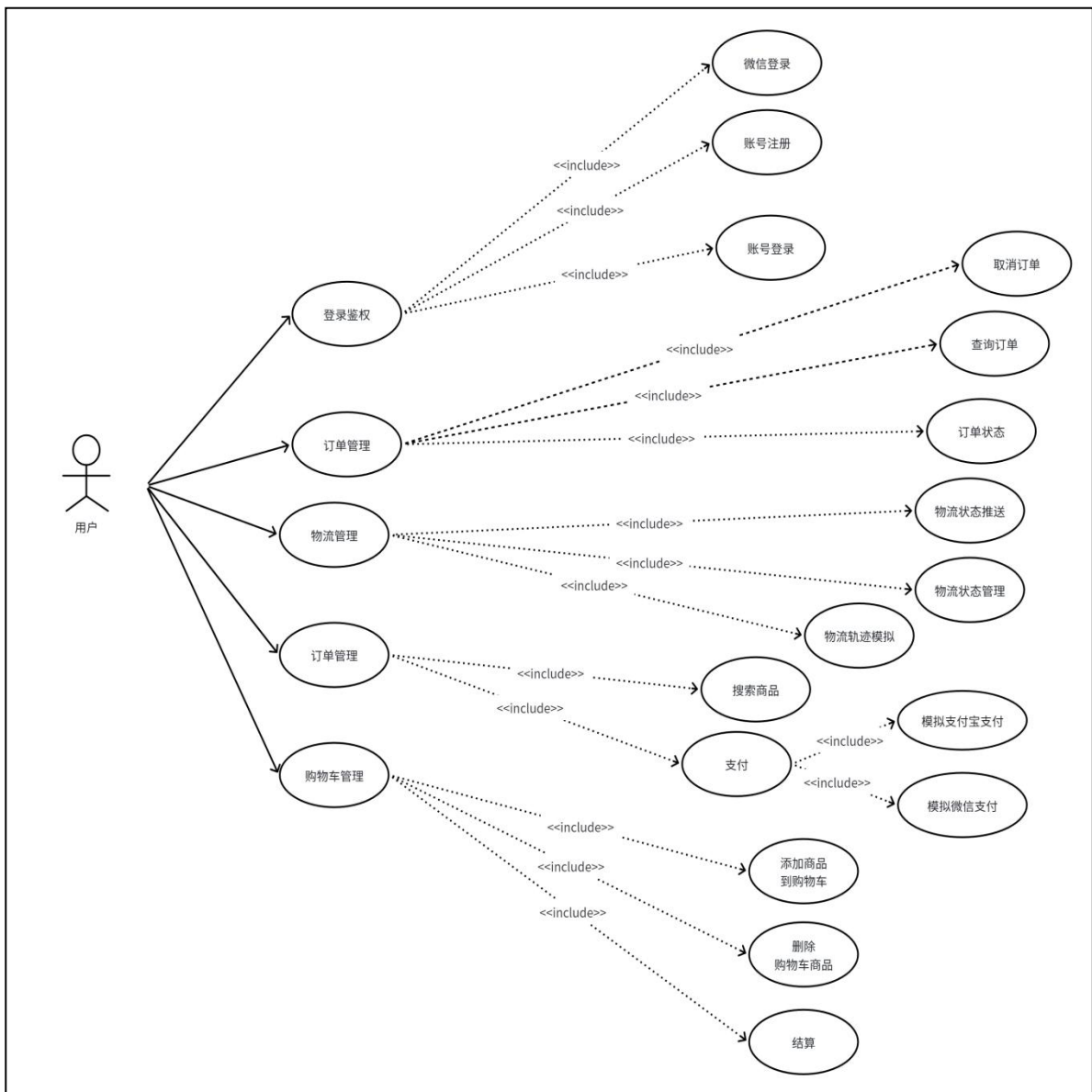


图 4 功能设计用例图

3.1.1 商品展示模块

在关于商品展示功能的实现过程之中，其作为用户选购商品最为基础的支撑环节，就必须能够把用户快速搜寻商品以及精准筛选商品的各类需求给切实地满足到了位；一是需要把商品分类展示的功能给落实好，从而方便用户可以根据自身实际的需求来更迅速地定位到心仪的商品；二是关于商品详情的展示环节，应当把包含商品图片、文字说明、规格参数以及销售价格在内的各项详细信息都给完整地呈现了出来，从而能为用户在最终选购商品的时候提供出一份较为全面的参考依据。

3.1.2 在线购物模块

在线购物是这个安琦购平台的核心业务模块，主要包含了用户从挑选商品到完成支付的整个流程，具体要做的功能如下：购物车功能，用户可以把看中的商品添加到购物车，还能对购物车中的商品进行批量管理，最后一键批量下单付款就行。在线下单这块，用户确认好要购买的商品后，就能在线提交订单，既可以填写新的收货地址，也能直接选择已有的地址，支付方式也能自主选择。第三方支付功能方面，整合了目前主流的第三方支付平台，支持微信支付、支付宝等多种付款方式，保证用户支付的时候既方便又安全。

3.1.3 物流跟踪模块

处于整个线上购物流程收尾处的物流追踪工作有其重要性，用户的整体售后体验常会直接被选购商品后的物流环节所左右；谈到具体功能的设计内容时，本系统把如下三项核心要点给涵盖了进去，一是能让物流运输状态实现实时可见，包裹流转到了哪个具体节点都能由用户随时去查看，二是把微信推送接口给接入进来了，只要包裹状态产生了变动就会有通知自动发过去，用户反复手动查询的累赘感能被有效减轻，三是在系统中有增设物流评价功能，真实的使用反馈被收集后，平台会把这类评价数据当作筛选物流服务商的一个准则来参考。

3.1.4 用户基础认证功能

依托微信小程序生态，支持微信快捷登录，无需复杂注册流程，快速完成用户身份验证，提升登录便捷性。

3.2 非功能性需求分析

非功能性需求直接影响平台的使用体验、稳定性与扩展性，结合微信小程序电商场景与技术栈特性，提出以下需求。

3.2.1 性能需求

关于整套系统所展现出来的性能表现到底是不是足够优异，往往是会直接地波及到了用户在实际操作过程当中那种整体感受，也正是因为这个缘故，在对技术架构进行搭建的完

整周期之内，明确地确定下了两个最为核心的优化方向：一方面是要把核心业务流程的响应速度给切实地保障好，特别是在涉及到商品检索、下单支付等关键页面的时候，把其响应的时

间给控制到了 0.5 秒以内的范围之内，以此来能让整套操作流程都变得要更加顺畅一些；二是要把业务数据实时对齐刷新的相关工作给真正地落实到了位，好让库存的实时变动、物流的具体进度等相关的信息内容，都能够及时且准确地呈现在用户的面前，从而能有效地减少那些因为数据未能对齐而给用户带来的不良体验。

3.2.2 易用性需求

小程序界面设计简洁直观，操作流程符合主流电商使用习惯，用户无需学习即可快速上手。操作提示清晰，异常情况如库存不足、支付失败实时反馈，引导用户完成操作，其中系统功能结构图如图 5 所示。

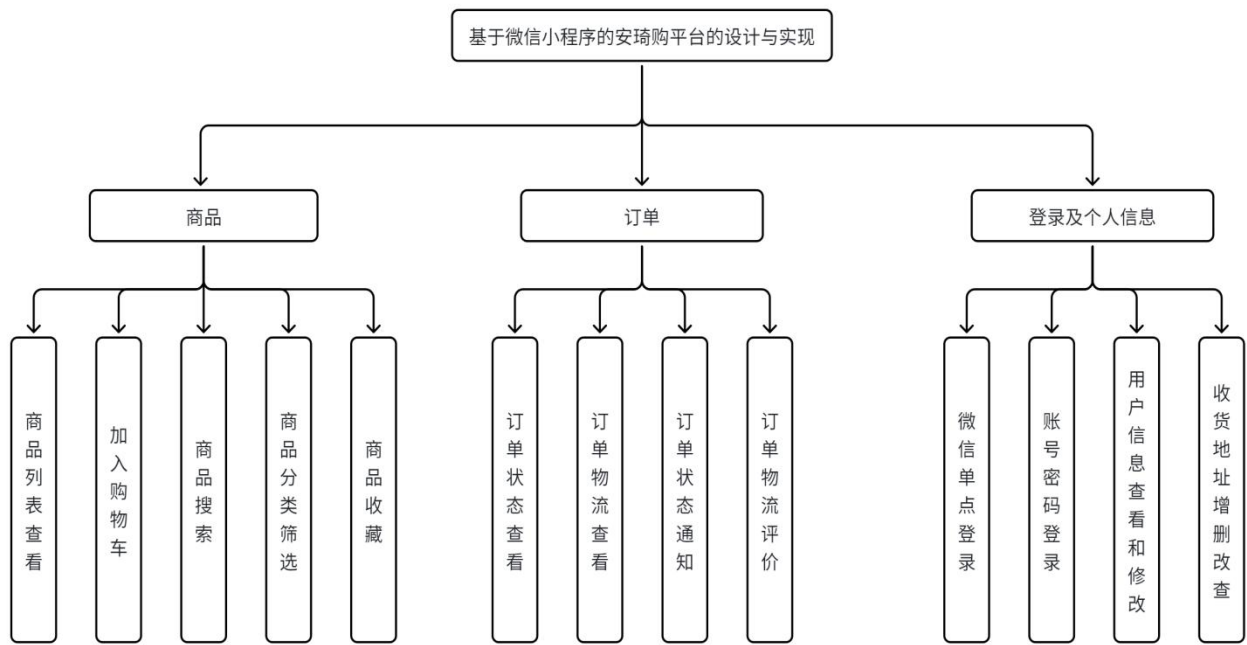


图 5 系统功能结构图

4 系统总体架构与数据库设计

系统总体架构设计是安琦购电商平台开发的基石，它的核心目标就是构建一个高内聚、低耦合、易于扩展且具备高并发支撑能力的分布式系统，本章会详细阐述系统的逻辑分层架构、微服务集群拓扑以及数据存储方案。

4.1 系统总体逻辑架构设计

在对这一整套系统进行整体性架构规划的那个关键阶段，其核心的环节主要就是运用到了这种纵向四层的解耦设计方式，整套架构已经被由上至下地清晰划分成了接入层、应用层、

领域层以及基建层这四个不同的维度，把各个功能模块原本那些模糊的职责界限给划分得更加清晰了；一是接入层会把所有需要跟用户直接产生交互的相关处理事务给承担起来，二是网关层能对那些汇聚而来的流量去实施统一的调配与管理，三是业务逻辑层会具体去实现核心功能里头的各类运算与数据处理工作，四是作为底层支撑的基建层也已经为数据的存放与持久化运行给提供到了最基础的保障；在把这样的设计思路给采用之后，即便是后续会有业务的更新或者是新增了一些功能组件，也只需要在对应的层级内部独立地开展拓展工作，以此来把整体架构运行的稳定性与后期维护的高效性给真正保障住了^[5]。

4.1.1 后端微服务集群拓扑架构

具体的那些微服务组件，还有它们各自对应的职责定义，都明确到了如下这几个方面，其中集群拓扑加购如图 6 所示：

(1) 在涉及流量守卫的控制环节里，系统会把 `anqigou-gateway` 这个 8080 端口给利用上，使其成了后端微服务集群的进出总门户；该网关除了有基础的请求转发功能，还把两项关键任务给承担了，一方面是负责路由与负载均衡的安排，让各类请求能顺着 `/api/product/` 这种前端传来的路径规则被分发到对应的微服务节点里；另一方面是把全局安全访问的管控给落实了，因为网关里集成了统一的验证逻辑，能够把没授权或者不合规的流量给拦在外面，既把公网和内部微服务间的隔离给做好了，系统整体的防护力也顺带着有了提高。

(2) 针对核心业务集群做的微服务化拆分，系统把本来就挺复杂的电商业务拆解成了多个能独立进行演进的微服务节点，一是用户服务模块实现了微信授权登录的相关功能，还能对用户的基础信息、收货地址还有收藏商品这些关键数据，进行统一的维护和管理，二是商品服务模块负责完成商品类别的划分和 SKU 规格的配置，同时还要保证能把数据和搜索引擎做到实时同步，这样就能让平台搜商品的时候，运行得既稳当又高效，三是搭建起了交易中心，靠着 Redis 的支撑，把高频的购物车逻辑给处理好了，还接管了从订单生成一直到售后结案的整个业务流程，四是把收银台做成了对外支付渠道的一种统一封装，专门用来处理那些异步回调里的幂等校验，五是把物流链路给建好了，能通过接口抓取到实时的物流轨迹，还会通过微信给用户发送物流推送的相关消息。

(3) 为了保障分布式架构里各个模块协同运转的效率，系统内部署了两个十分核心的支撑组件，一是服务治理中心的 Nacos 集群，它主要通过配置文件实现全局配置的动态推送，同时完成微服务的自动注册和服务发现等工作，保障各服务节点之间的正常通信，二是采用了远程调用框架 OpenFeign，以此实现微服务模块之间的内部数据通信，就拿订单的业务场景来说，订单服务能够通过接口直接调用到商品服务，从而顺利完成库存扣减这类跨服务的

业务操作。

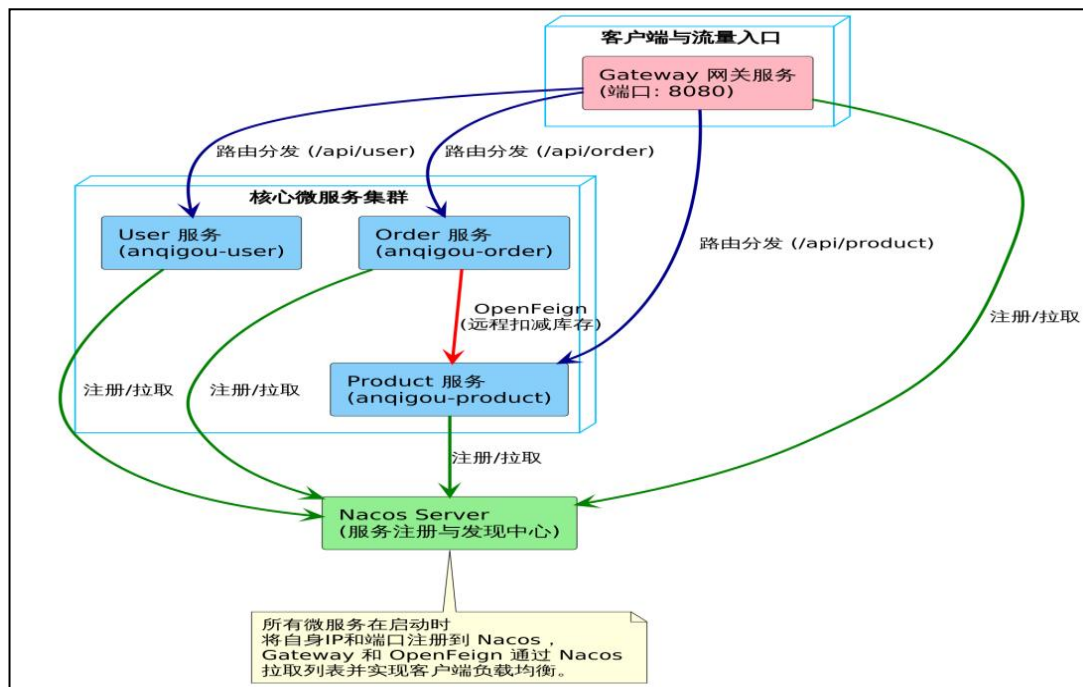


图 6 后端微服务集群拓扑图

4.1.2 前后端分离的数据交互流程

为了把项目的整体开发效率给提升上来，该系统在设计时选用了前后端分离的架构模式，前端主要承担页面的渲染和跟用户之间的交互操作，后端则把精力专注在业务逻辑的处理还有对数据的存储和读取上，这两者之间依靠标准的 RESTful API 来完成数据交互的工作，在把接口规则统一约定好了之后，大幅度缩短了项目的整体研发周期了。

系统还专门针对数据交互的格式还有通信的流程制定出了较为清晰的规范标准，所有的交互操作都用到了 HTTPS 协议，数据在传输时也统一采用了 JSON 的格式，为了把前端的数据处理压力减轻，后端设计出了通用的响应类 `ApiResponse<T>`，对 `code` 状态码、`message` 提示信息还有 `data` 业务数据这些内容都做了标准化的封装，这样前端就能根据状态码直接执行对应的业务处理逻辑了。

商品服务在接收到请求之后，会先去查询 Redis 的缓存，要是缓存里没有对应的数据，就会去访问 MySQL 数据库，最后把数据封装成 `ProductDetailDTO` 对象给返回出来，前端的 Vue 3 框架在接收到 JSON 的响应数据之后，会利用响应式的特性完成页面的更新和渲染，以此实现数据展示的一整套流程，系统架构图如图 7 所示。

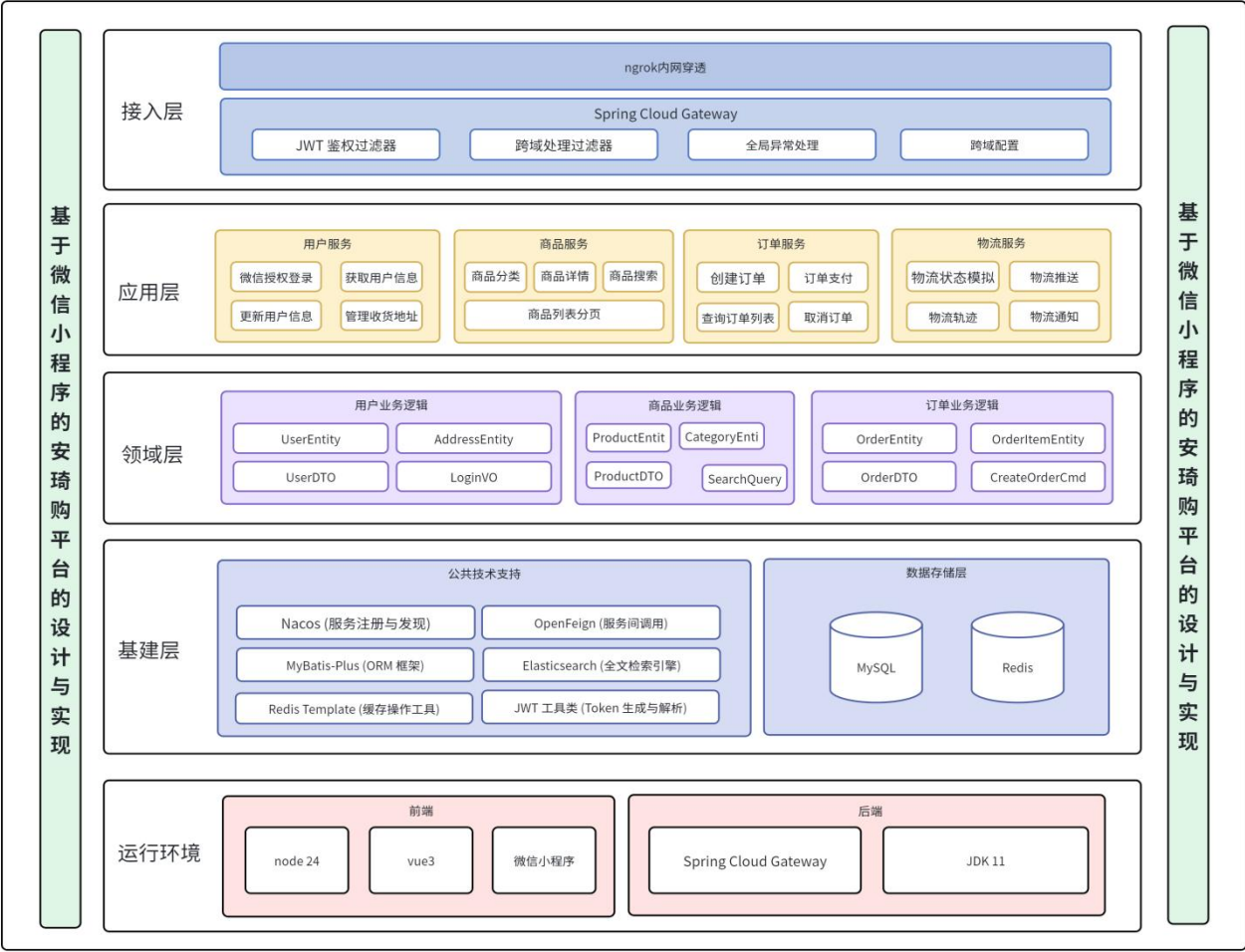


图 7 系统架构设计图

4.2 关系型数据库逻辑设计

这套系统选用 MySQL8.0 作为核心关系型数据库，主要存储用户信息、商品交易、物流轨迹等结构化、关联性强的数据，数据库设计的时候主要遵循第三范式，尽可能降低数据冗余，同时结合电商业务的实际需求做了适当的反范式设计，比如在订单表中冗余收货地址信息，这样能减少多表关联查询的次数，把系统的查询效率给提了上来^[6]。

4.2.1 数据库概念模型设计(E-R 图)

根据业务需求的梳理分析，提取出了系统的核心实体及实体间的关联关系，如图 8 所示，最终形成了数据库的概念模型，具体内容如下：

用户实体 (User) 作为系统的核心主体，一个用户可以对应多个收货地址，同时能够创建多个订单，也可以收藏多款商品，与商品为多对多的关联关系。

商品实体 (Product) 归属于单一商品分类，一个商品会包含多个不同规格的 SKU 信息，比如颜色、尺码等，同时商品可以被添加到多个订单的订单项中。

订单实体 (Order) 由用户创建，一个用户可以创建多个订单，每个订单包含多个订单项，

且一个订单仅对应一条支付记录和一张物流单据。

物流实体(Logistics)与订单一一对应，一张物流单中会包含多条连续的物流轨迹记录。

售后实体(AfterSale)主要关联订单内的指定商品项，由用户发起申请，再由商家进行后续处理。

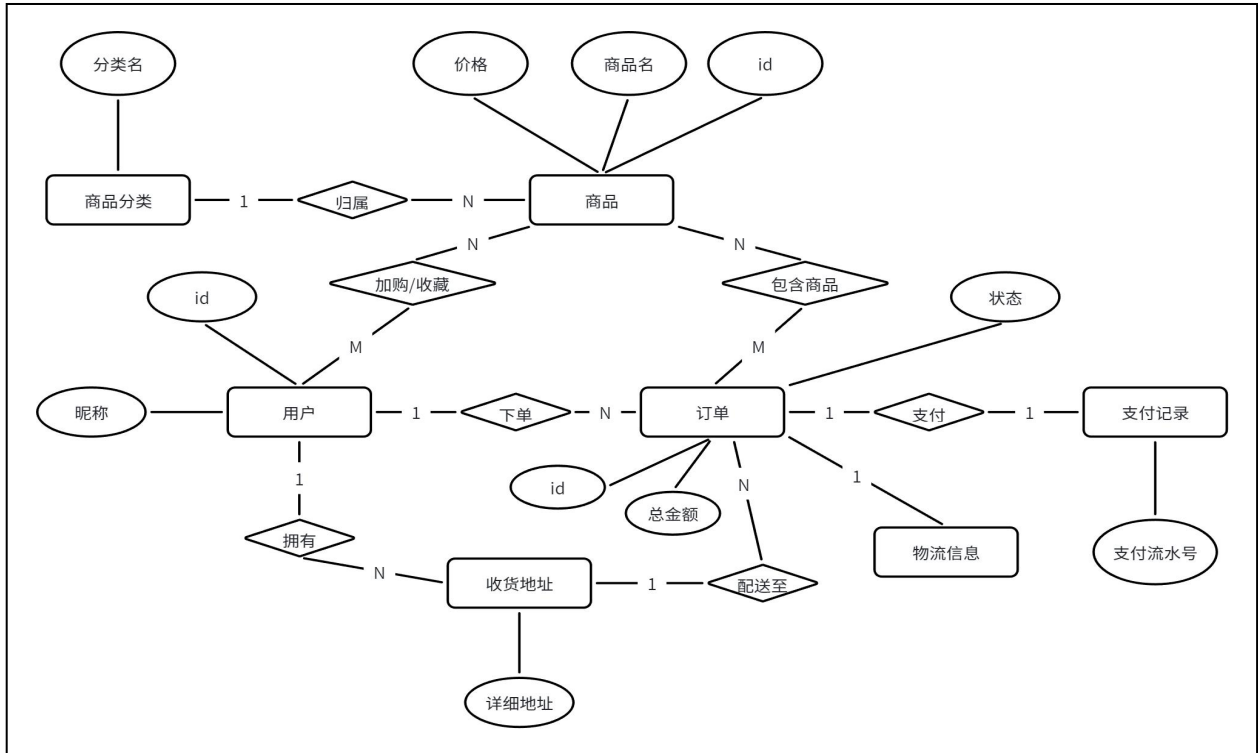


图 8 系统整体 E-R 关系图

4.2.2 核心业务数据表结构设计

本系统共包含 16 张业务表，以下列举最核心的 5 张数据表结构设计。

(1) 用户信息表(user)

记录用户的基本信息及微信授权信息，phone 和 wechat_open_id 均设有唯一索引以确保账号唯一性，如表 1 所示。

表 1 用户表

字段名	类型	长度	约束	是否非空	描述
id	varchar	36	PK	是	用户主键 ID
phone	varchar	11	Unique	是	手机号(登录账号)
nickname	varchar	50	NN	是	用户昵称
wechat_open_id	varchar	100	Unique	是	微信 OpenID
member_level	tinyint	4	-	是	会员等级(0: 普通)
status	tinyint	4	-	是	状态(0: 正常 1: 禁用)

(2) 商品主表(product)

存储商品的基础信息，description 字段采用 longtext 存储富文本详情。stock 为商品总库存，实际交易扣减的是 product_sku 表的库存如表 2 所示。

表 2 商品主表

字段名	类型	长度	约束	是否非空	描述
id	varchar	36	PK	是	商品主键 ID
category_id	varchar	36	FK	是	关联分类 ID
name	varchar	200	NN	是	商品名称(全文索引)
price	bigint	20	NN	是	销售价格(单位：分)
stock	int	11	-	是	商品总库存
status	tinyint	4	-	是	上架状态(0：下架 1：上架)

(3) 订单主表(orders)

存储订单维度的核心数据。为了便于历史追溯，表内冗余了 receiver_address 等收货快照信息，防止用户修改地址后影响历史订单显示，如表 3 所示。

表 3 订单主表

字段名	类型	长度	约束	是否非空	描述
id	varchar	36	PK	是	订单 ID
order_no	varchar	20	Unique	是	业务订单号
user_id	varchar	36	FK	是	下单用户 ID
total_amount	bigint	20	NN	是	订单总金额(分)
status	varchar	20	-	是	状态(pending_payment, paid, shipped. . .)
create_time	datetime	-	-	是	下单时间
remark	varchar	255	-	否	用户下单备注

(4) 订单详情表(order_item)

存储订单包含的具体商品行。sku_id 关联具体的规格，unit_price 记录下单时的商品单价，确保价格波动不影响已生成的订单，如表 4 所示。

表 4 订单详情表

字段名	类型	长度	约束	是否非空	描述
id	varchar	36	PK	是	详情 ID
order_id	varchar	36	FK	是	关联订单 ID
product_id	varchar	36	FK	是	关联商品 ID
product_name	varchar	200	NN	是	商品快照名称
spec_info	varchar	500	-	是	规格描述(如：红色，XL)
quantity	int	11	NN	是	购买数量

(5) 物流信息表(logistics)

用于存储快递发货信息。系统通过 tracking_no(快递单号)对接第三方物流接口，status 字段用于同步最新的物流流转状态，如表 5 所示。

表 5 物流信息表

字段名	类型	长度	约束	是否非空	描述
id	varchar	36	PK	是	物流 ID
order_id	varchar	36	FK	是	关联订单 ID
courier_company	varchar	50	NN	是	快递公司名称
tracking_no	varchar	100	Unique	是	快递单号
status	varchar	20	-	是	状态(shipped, signed, exception)

4.3 搜索引擎与缓存架构设计

MySQL 虽然能保证数据的 ACID 特性，适合存储结构化数据，可在面对海量商品的全文检索和高并发的热点数据访问时，很容易就会成为系统的性能瓶颈，所以在数据基础设施层引入了 Redis 分布式缓存和 Elasticsearch 全文搜索引擎，用“空间换时间”的思路，大幅提升了系统的吞吐量和响应速度，让用户的使用体验变得更流畅^[7]。

4.3.1 Redis 分布式缓存策略与数据结构设计

开发中主要用 Redis 存储热点商品数据、临时验证码和用户的鉴权状态，为了解决 Redis 默认序列化的问题，还在 Spring Boot 环境中自定义了 RedisConfig 配置类，对序列化机制做了深度优化。实际业务中，把高频访问且极少变动的数据，比如商品分类树、首页轮播图，用 Hash 或 String 结构缓存起来提升访问速度，同时利用 Redis 的 TTL 过期特性，实现了用户 JWT Token 的黑名单机制和登录验证码的自动过期，比如验证码设置 5 分钟过期，能避免用户长时间不操作导致的安全问题。

4.3.2 Elasticsearch 索引映射(Mapping)设计

为了实现商品名称模糊搜索、多维度条件筛选这类复杂功能，系统在 Elasticsearch 里创建了专属的商品索引 product_index。对应的索引映射(Mapping)规则，都在 ProductDocument 实体类中完成定义，如图 9 所示，核心设计主要分为三个部分：

(1) 字段类型与分词设置：对于商品名称、品牌、描述这类长文本内容，设置为 text 类型，搭配 standard 分词器来实现全文检索；同时额外添加 keyword 子字段，用来满足数据聚合、精准排序的要求。而分类 ID、商家 ID 这类唯一标识字段，统一使用 keyword 类型，不做分词处理，保证条件过滤的精准度。

(2) 索引存储空间优化：主要是把商品主图 URL 这类本身就不会参与到检索环节里的字段，在进行 Mapping 配置的过程中把 index 属性设成了 false，这样就能让 Elasticsearch 只把这些数据给保存下来而不会去创建对应的倒排索引，也就能把系统的内存还有磁盘的占用给有效地节省下来了。

(3) 数值和排序相关字段规划：把商品的售价以及原价都统一用了 long 类型来进行存储，这里是把分当作了计价的单位来计算的，这样就能避免掉浮点数在计算的时候会出现的精度上的误差，另外还把销量用 integer 类型、评分用 double 类型来存储，也能为后续的价格区间筛选或是多维度排序这类的功能，打下一个比较好的底层数据方面的基础。

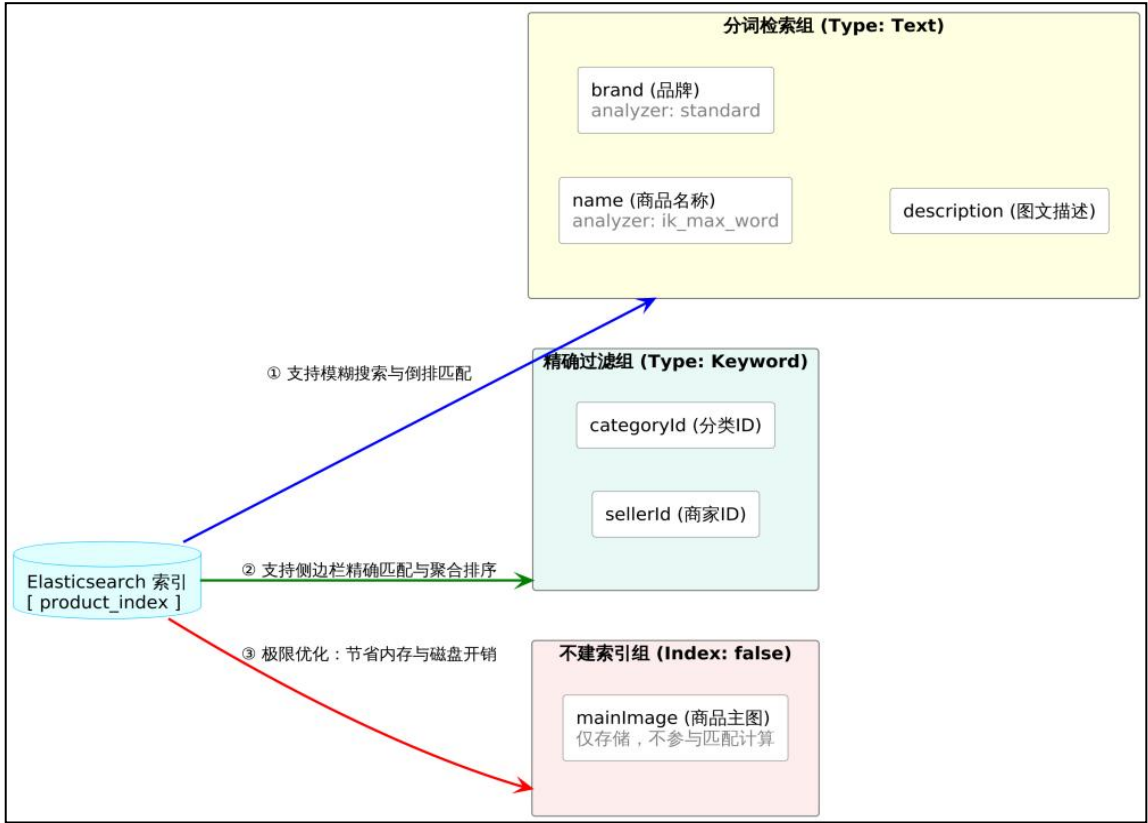


图 9 Elasticsearch 商品索引结构图

5 核心功能模块的详细设计与实现

本章作为安琦购平台业务落地核心，依托前期设计完成五大模块的详细设计与实现，将逐一说明五大模块实现方案，呈现电商业务全闭环的开发过程。

5.1 用户登录模块设计与实现

用户登录是安琦购平台的身份核验与安全入口，依托微信小程序生态实现快捷登录，结合微服务架构与 JWT 无状态认证完成全流程鉴权，保障登录流程简洁、安全且适配分布式系统部署要求。

5.1.1 前端微信授权登录与状态管理实现

关于在前端整体设计与开发工作里，利用了 uni-app 并且搭配着 Vue3 框架来把相关的开发事宜给全部落实好了的，同时也已经把微信小程序里那套原生的授权机制给同步配套地搭建到了位；就在用户把登录那个按钮给点下去了之后，程序内部就会把 uni.login()这种方法调

用，从而能够拿到由微信平台所提供的临时登录凭证 `code` 码，通过把封装好的网络请求库给利用起来，从而把这个 `code` 码给准确地发送到后端，登录注册模块如图 10 所示。

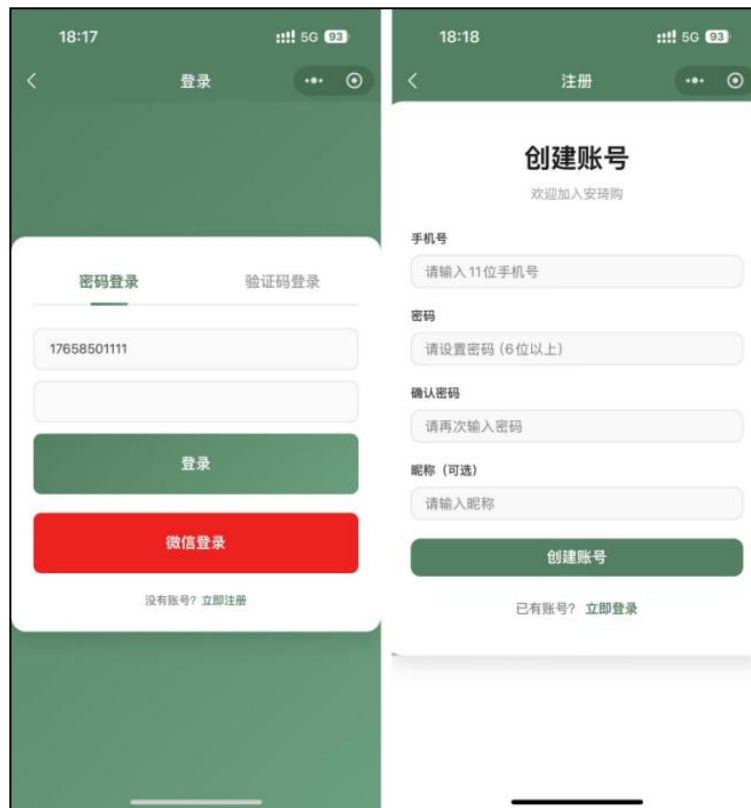


图 10 登录注册模块

5.1.2 后端认证签发与网关统一鉴权实现

在后端的用户微服务接收到了前端发送过来的 `code` 码之后，就会去调用微信开放平台那边的接口来把用户专属的 `OpenID` 给解析并拿到手，并且能以这个 `OpenID` 作为依据来把用户的自动注册或者信息查询给办理好；在认证通过了之后，系统会采用 `HS256` 这种算法来把 `JWT Token` 给制作出来，并把用户编号、过期时间等信息都给填入到数据载荷里头，而加密钥匙和有效时间则是通过 `Nacos` 配置中心来开展统一的管理。

5.2 前端交互与状态管理实现

安琦购平台的前端基于 `uni-app` 和 `Vue3` 搭建，在多端适配的电场景里，跨组件的状态共享和频繁的网络请求是开发中遇到的两个核心问题，本节会具体说明系统如何通过 `Pinia` 解决全局状态管理的问题，也会介绍如何封装统一的网络请求库，来处理接口鉴权和异常响应的问题，让前端的开发和维护更高效。

5.2.1 基于 `Pinia` 的全局状态管理实现

之前做 `Vue2` 项目时一直用 `Vuex` 做状态管理，实际用下来发现 `mutations` 的设计太繁琐，

代码的冗余度特别高，这次开发用 Vue3 的组合式 API，搭配 Pinia 就特别适配，代码的时候更简洁，对 TypeScript 的支持也更好，多人协作开发时能减少很多类型错误^[8]。

在 actions 里封装了 login 和 logout 两个核心方法，用户登录成功后，不仅会更新内存中的 token 和 userInfo，还会通过 uni.setStorageSync 把数据持久化到本地存储；用户退出登录时会同时清除内存和本地的状态数据。这样的设计让前端的状态流转特别清晰，用户修改个人信息、切换登录状态时，只需要调用 Store 里的 action 方法，全站依赖这些状态的组件就会自动响应式重新渲染，实现了单向数据流，后续维护的时候也能快速定位问题。

如图 11 所示，图中所展示的是“安琦购”微信小程序的首页以及商品搜索结果页。

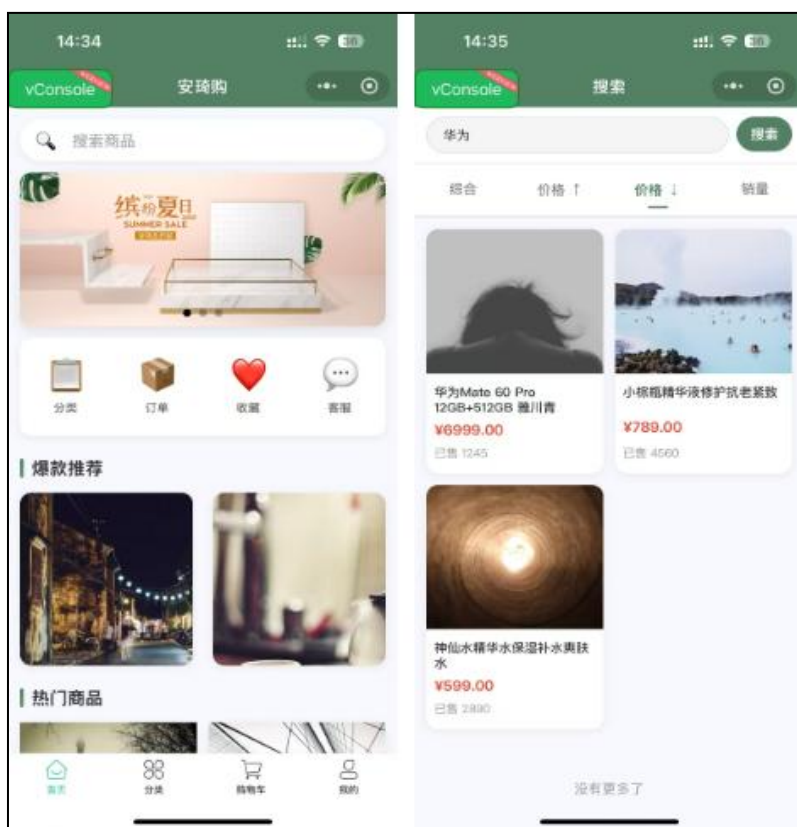


图 11 平台首页及商品全文检索结果展示

5.2.2 Axios 网络请求封装与拦截器实现

uni-app 本身就提供了 uni.request 这一原生网络请求 API，可这个 API 在多端适配的过程中，并没有统一的拦截器机制；要是在每个页面调用接口的时候，都手动去处理 Token 携带、异常状态码这类逻辑，不仅会让代码的重复率变得很高，还容易出现处理遗漏的情况，进而导致鉴权失败、异常未处理这类问题的出现。所以就在 src/utils/request.ts 这个文件里，对 uni.request 做了二次封装处理，还模拟实现了 Axios 的请求与响应拦截器，把鉴权、异常处理这类通用的逻辑都抽离了出来^[9]，这样一来，业务页面就只需要去关注接口的调用以及返回数据的使用就可以了，设计流程如图 12 所示。

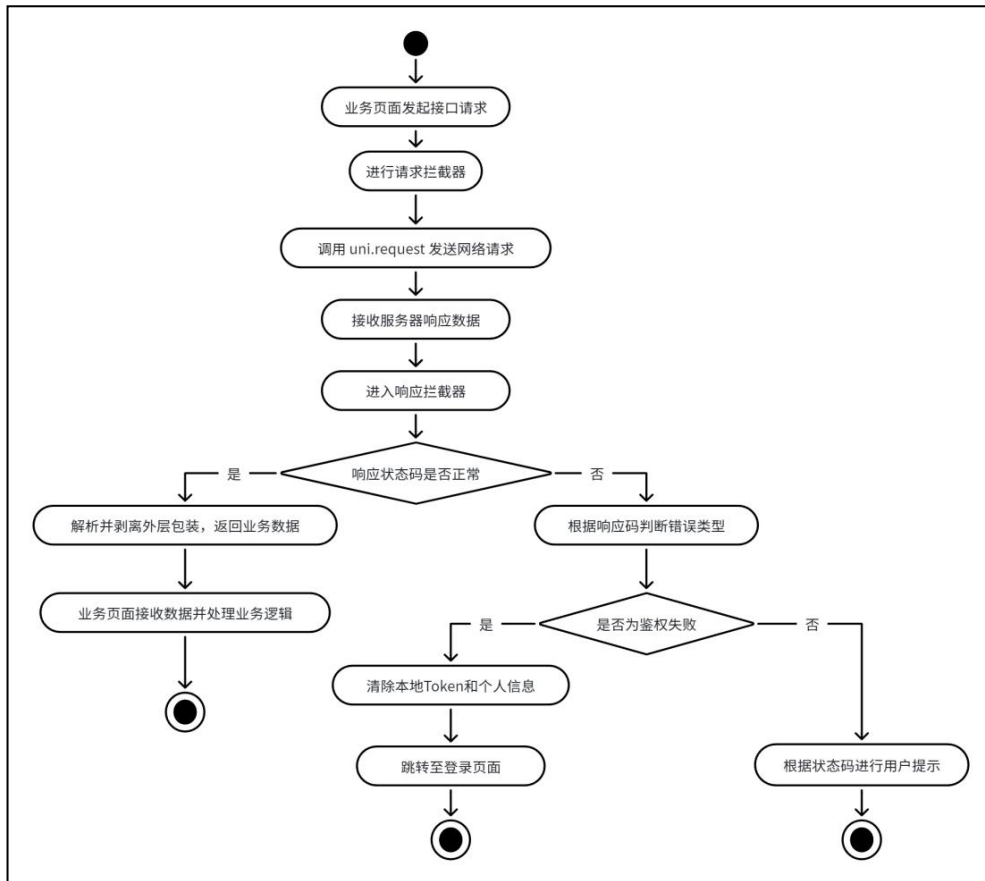


图 12 前端网络请求与拦截器处理流程图

5.3 商品检索与渲染模块实现

商品检索与展示功能，是电商平台的核心流量入口，同时也是用户平时使用得最频繁的模块，用户对于这个模块的核心需求，其实就是搜得快、看得清、选得准，说得更具体一点，就是检索的响应速度要足够快，商品的信息展示要全面完整，多规格的 SKU 选择操作要顺畅不卡顿。

5.3.1 商品数据同步与 ES 高性能检索实现

在解决 MySQL 面对海量数据背景下处理商品模糊查询时暴露的性能不佳缺陷时，本系统采用了“由 MySQL 充当主要存储、Elasticsearch 作为检索辅助”的异构存储结构，并将其引入到系统开发过程之中，具体的实施路径是，先将商品数据存储到 MySQL 数据库里，再借助定时任务这种异步同步的方法，把数据同步到 Elasticsearch 的索引当中；只要商品的相关信息发生过更新或是变动，系统内部就会自动同步修改 Elasticsearch 里的对应索引，以此来始终保住两边数据在逻辑层面的高度一致性，而在开展真实的检索业务流程时，后端结合着 Elasticsearch 与 IK 分词器，将关键词的模糊匹配以及多条件的筛选任务落实到位；由于已经实践了这种设计方案，哪怕是在万级数据量的业务场景下，单次查询的响应时长也能够

控制在 100 毫秒以内，这不仅显著提升了系统的检索效率，同时也优化了用户在实际使用过程中的整体体验。

5.3.2 商品详情渲染与多规格 SKU 集成

商品详情页是促成用户下单的关键页面，除展示商品轮播图、富文本详情、价格、库存等基础信息外，核心交互为多规格 SKU 选择。该功能的核心难点是规格组合的逻辑处理，手动编写笛卡尔积算法耗时较长，还容易造成移动端性能卡顿，商品详情页及 SKU 模块如图 13 所示。

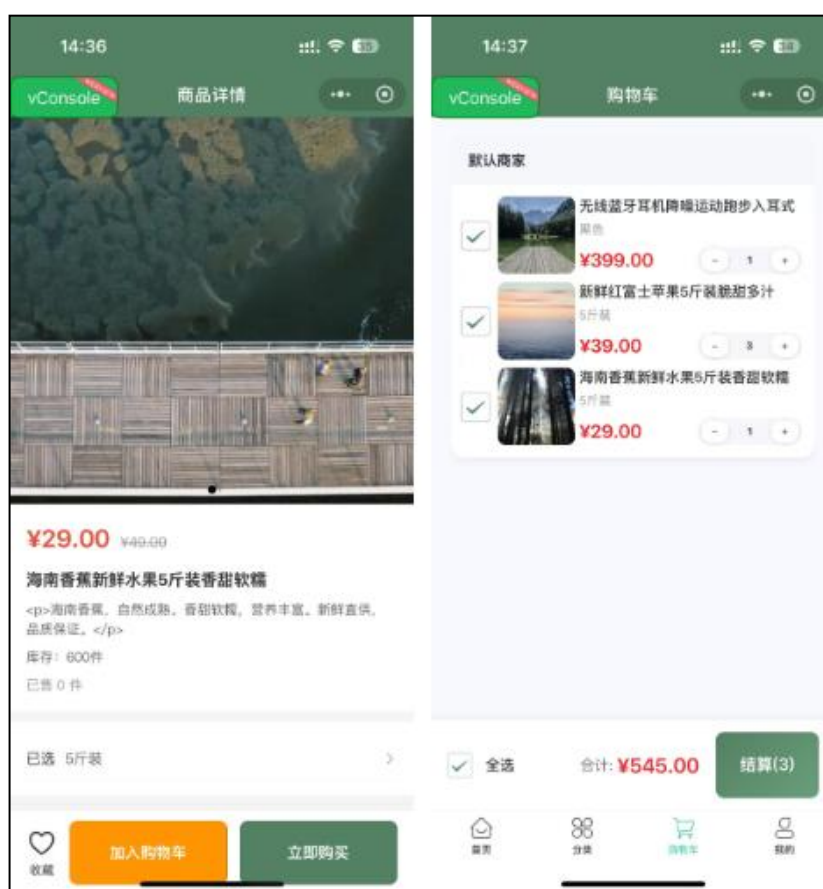


图 13 商品详情页及多规格属性选择面板

为了能够减少重复开发的工作量并将现有组件充分利用，项目从插件市场引入了 geek-sku 组件，也将开发重心放到了数据调优与交互接管的工作上；一方面后端所提供的 SKU 列表为平铺格式，和组件适配标准存在出入，编写适配器函数对数据做重新解析，将处理完成的数据传入组件过后，便能顺利化解相关的适配难题，对应的具体流程可见图 14。



图 14 商品检索与详情展示时序图

5.4 订单交易与支付模块实现

订单交易与支付其实算得上电商平台具备核心价值的商业闭环板块，也会是项目开发过程里实现难度相对偏高的功能模块；该模块会涉及到跨服务之间的数据一致性保障工作、高并发运行环境下的库存数量核减，还有模拟支付接口的安全连通等诸多核心难题，流程当中只要有任意一处出现细节疏漏，就会扰乱整体交易业务原本的正常运行节奏，事态如若发展严重，还会引发资金损耗以及库存数据紊乱丢失这类潜在隐患。在参与实际项目开发的过程当中，也针对这类真实存在的业务痛点完成了专项设计与优化调整，一是依靠瘦数据模型将购物车内部的各类相关数据完成标准化的管控梳理，二是借助乐观锁的底层运行逻辑去落地分布式应用场景当中的库存扣减业务流程，三是运用配置与业务相互拆分离的方式对接连通模拟支付接口，也很好地维系住了整条交易链路的平稳运转以及完整的安全防护标准。

5.4.1 购物车管理与 Redis 存储实现

在实际开发过程中，采用了“瘦数据模型”的方式来设计购物车表(shopping_cart)，表里面只存了 user_id、product_id、sku_id 和 quantity 这四个核心的关联关系，并没有存储商品名称、价格这类容易发生变动的信息，这样做不仅能减少数据库的存储压力，还能避免出现数据冗余的情况。当用户打开购物车页面的时候，订单服务就会触发 getCartItems 方法，它不会直接返回数据库里面的静态数据，而是通过 Java8 StreamAPI 去遍历购物车的相关记录，再借助 OpenFeign 远程调用商品服务，获取到 SKU 的最新售价、库存以及商品上下架状态，然后实时组装好数据并同步到前端页面，这样就彻底避免了购物车出现脏数据的问题，这也是实际开发过程中保证购物车使用体验的关键所在。

从图 15 能看到，这两个界面分别是安琦购小程序的购物车页面和提交订单页面。

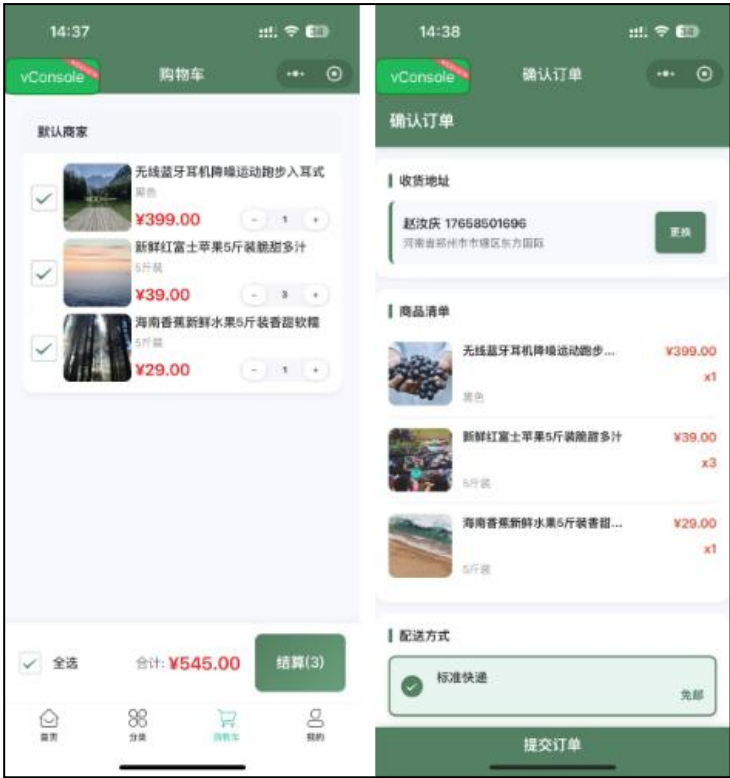


图 15 购物车聚合展示及提交订单视图

5.4.2 订单全生命周期状态流转与库存扣减

订单从创建到最终完成，会经历多个不同状态的流转过程，在 `orders` 表里面专门设计了 `status` 字段，还定义了一套标准的状态机流转模型：`pending_payment`(待付款)→`paid`(已付款，待发货)→`shipped`(已发货，待收货)→`completed`(已完成)，而且每个状态的切换，都会有对应的业务逻辑去触发，这样就能避免状态出现混乱的情况，还能方便后续的订单管理工作和数据统计工作^[10]。

核心的下单逻辑里面，最关键的环节就是分布式的库存扣减操作，要是在高并发场景下处理得不够好，就容易出现库存超卖的问题，这可是电商平台开发过程中的大忌。在实际开发的时候，专门做了多层校验来规避这些问题：一是在前端做库存的校验工作，防止用户恶意提交超出库存数量的订单；二是后端接收到下单请求之后，会再次实时拉取所选 `SKU` 的最新价格，重新计算出订单的总金额，避免前端恶意篡改提交的金额数据；三是库存扣减采用乐观锁的方式，商品服务会执行 `UPDATE product_sku SET stock=stock-#{quantity} WHERE id=#{skuId} AND stock>=#{quantity}` 这条 SQL 语句，只有当库存数量大于等于用户购买的数量时，才会执行库存扣减的操作，这样就能确保高并发场景下不会出现库存超卖的情况，在实际测试的时候，模拟了数十个并发下单的请求，库存的扣减操作都做到了精准无误^[11]。

库存锁定成功后，系统会将订单主表和订单项表的数据持久化到数据库，再异步清理用

户购物车中已经结算的商品项，完成整个下单的流程，设计思路如图 16 所示。

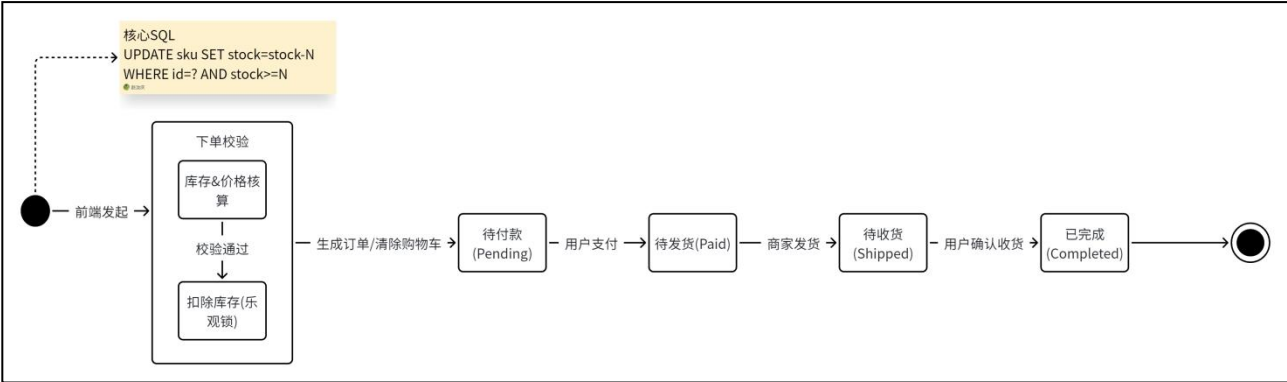


图 16 订单创建与库存扣减状态图

5.4.3 模拟微信/支付宝第三方支付接口对接

本系统的支付模块是接入到了模拟支付的渠道里头的，以此来满足好开发还有测试这些环节当中的支付校验工作；在去把代码给实现出来的过程当中，是把核心的参数都统一存放在了 SimulatePayConfig 这种配置类里面，这样就能把配置的内容和具体的业务逻辑给相互分离开来，也能方便后续对代码做维护，还能方便进行功能的拓展；等订单服务把相关数据提交到支付模块之后，这个模块就会自动生成模拟交易的凭证，还会把凭证回传到前端页面去；一是为了能把第三方支付那种真实的回调逻辑给还原出来，系统采用了本地内部调用的方式来触发异步回调，在回调的过程当中还会去执行对应的安全核验操作，二是只有等数据校验合格之后，才会去更新支付状态，还会发送订单状态变更的相关通知，这样就能确保支付的整个流程能形成一个完整的闭环，而且运行起来也是挺稳定可靠的^[12]。

5.5 物流跟踪与消息推送实现

物流信息的透明度直接影响用户的购物体验，用户下单后都希望能实时看到商品的物流轨迹，还能及时收到物流状态的变更通知。但实际开发中，个人开发者没有企业资质，无法对接快递 100 等第三方物流的正式接口，所以设计了物流轨迹的动态生成算法，能模拟出逼真的物流轨迹，同时结合微信小程序的订阅消息功能，实现了物流状态的实时推送，打造了完整的物流跟踪与消息通知体系。

5.5.1 第三方物流轨迹查询接口封装

为了在没有真实物理物流支撑的情况下，依然能向用户展示逼真的快递流转过程，比如“已揽收”，“转运中”，“派件中”，系统内部实现了一个高仿真的物流数据模拟器 LogisticsTrackGenerator，当用户首次查询某个订单的物流详情且数据库中无相关记录时，系统会触发这个生成引擎。

物流轨迹生成算法的实现核心分为三个环节：

在路线规划的相关操作环节里，系统会先把订单内标注的收寄城市信息给读取出来，再通过调用 `generateTransitRoute` 这一方法，结合国内当下的主干物流网络对中间会途经的节点做计算；以从深圳发往上海的订单为例的话，程序会依据地理运输的路线，把广州、长沙还有武汉这些中转站给自动添加进去，这样能让模拟出的物流路径更贴合实际的干线运输规则。

在节点时间推演的部分，会结合物流流转在时序上的要求，系统会用到 `LocalDateTime` 类，以当前的时间为基准把时间往后递推；为了还原出真实的网点作业流程，代码里会通过随机变量对各个环节的处理时长做设定，比如把转运中心分拣货物的时长随机设置在 35 到 55 分钟的区间内，进而生成出符合实际逻辑的时间序列记录。

在业务信息动态组合的环节，为了进一步提升测试数据的真实性，系统内置了包含快递员姓名、车牌号以及客服工号在内的基础数据集，会通过 `String.format` 函数把随机的参数动态填充到文本模板当中；借助这一套实现方案，系统能在未获取第三方物流 API 授权的离线开发环境下，自动生成结构完整且逻辑合理的物流流转数据，把前端界面展示和后期压力测试时的数据使用需求给有效满足掉。

如图 17 呈现出来的物流轨迹查询页面，这个页面主要被分成了地图展示与文字节点记录。



图 17 物流轨迹地图轨迹视图及轨迹文字描述

5.5.2 基于微信订阅消息的物流状态实时通知

为了实现发货或是签收这类物流关键节点的用户主动通知，本系统接入了微信小程序的订阅消息功能，在具体的编码实现阶段主要攻克了两项技术方面的难点：

第一点是 **AccessToken** 的获取和它的缓存策略，考虑到微信接口存在调用频次的限制，且推送消息时必须带上有效的凭证，所以项目中用 **Redis** 对 **Token** 做统一的缓存；程序运行时优先从 **Redis** 里读取 **Token**，若发现缓存已过期或是不存在，就会向微信服务端请求新的 **Token**，拿到后再将其存入 **Redis** 并把过期时间设为 7000 秒；这一方案既规避了因频繁请求引发的限流问题，也减少了推送环节里的网络耗时。

第二点是消息内容的拼装和跳转路径的配置，后端会依据已通过微信审核的模板格式，把订单编号和物流状态等参数动态填到对应的字段里，订单状态和订阅通知如图 18 所示；同时，推送的请求中会配置好目标页面的路由，用户点击微信推送的卡片后，就能直接跳转到小程序内的订单详情页，具体的订阅消息推送流程可参考图 19。



图 18 订单状态及微信订阅通知

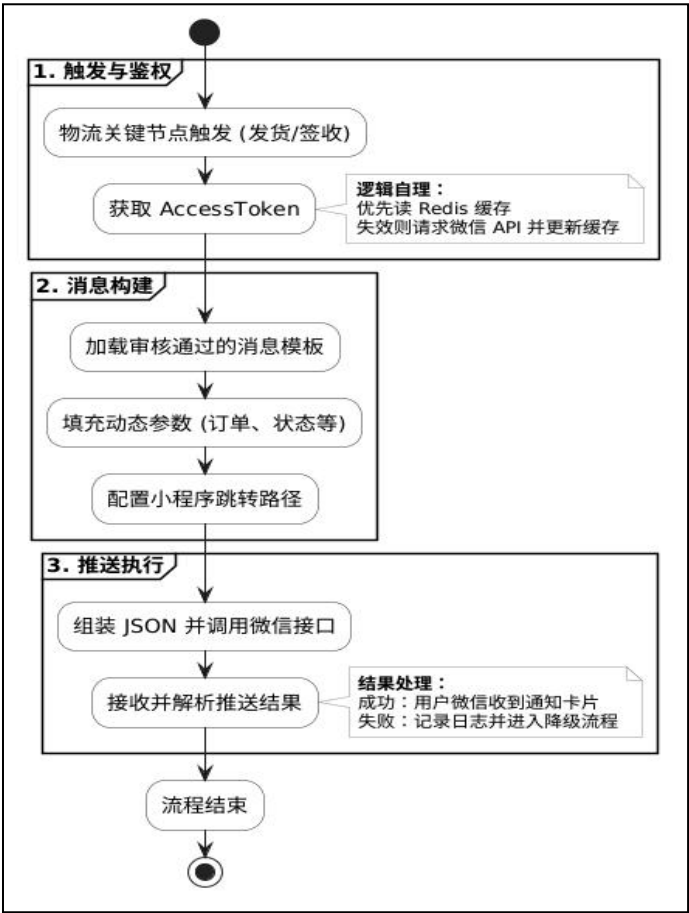


图 19 微信订阅消息推送业务处理逻辑图

6 系统测试与性能优化

为了让安琦购平台能在日常运行中始终保持稳定且高效的状态，本研究把系统核心功能验证与用户交互感官评估给结合了起来，通过黑盒测试的方法把从登录、选购到支付维权的闭环业务流程都给全覆盖到了，与此同时，针对电商平台读多写少的特征把优化工作给落实到了架构层面，一是以配置缓存规则的方式把数据库的压力给有效分担了，二是在对索引与查询语句做出调整后，数据的调取效率能有明显的进步，三是把核心服务进行了集群化部署并配上了负载均衡机制，使得系统应对高并发的承载能力给得到了实质性的增强，正是由于这一连串测试与优化动作的落地，平台的响应灵敏度与运行稳健性才有了质的飞跃^[13]。

6.1 系统功能测试

为了能把系统的稳定性和可靠性给彻底地检验出来，本次测试是专门去采用了黑盒测试里头最常用的等价类划分，还有那个边界值分析法来开展的；在具体的执行过程当中，是要求测试人员得去完全地模拟出用户在现实生活里的那种实际操作行为，从而对平台里头像是注册登录、商品浏览检索、订单交易支付，还有物流轨迹查询这些个最核心的业务主链路，都给逐一地做过了非常全面的交叉测试；也为后续系统的正式上线去打下了一个挺扎实的技

术基础^[14].

以下为安琦购平台核心功能模块的测试用例及测试结果统计表:

(1) 用户认证测试案例表如表 6 所示.

表 6 用户认证测试案例表

用例编号	测试模块	测试项	操作步骤描述	预期结果	实际结果	测试结论
User-01	用户认证	微信授权登录	在小程序端点击“微信一键登录”，允许授权.	系统自动获取 OpenID, 若为新用户则静默注册并返回 JWT 令牌, 跳转至首页.	成功获取令牌并跳转首页.	通过
User-02	用户认证	Token 过期拦截	手动清除本地 Storage 中的 Token, 点击“购物车”页面.	网关层拦截请求, 前端拦截器识别 401 状态码, 强制跳转至登录页.	弹出“登录已过期”, 并重定向至登录页.	通过

(2) 商品模块测试案例表如表 7 所示.

表 7 商品模块测试案例表

用例编号	测试模块	测试项	操作步骤描述	预期结果	实际结果	测试结论
Product-01	商品模块	关键词模糊检索	在搜索框输入关键词“手机”，点击搜索.	触发 ES 检索引擎, 毫秒级返回标题或描述中包含“手机”的商品列表.	列表正确渲染相关商品, 响应无明显延迟.	通过
Product-02	商品模块	多规格 SKU 选择	进入商品详情页, 点击“加入购物车”，选择“红色”+“256G”规格.	自动匹配并高亮对应的 SKU, 动态显示该规格的真实库存与售价.	价格与库存根据选中规格动态切换, 缺货项置灰.	通过

(3) 订单模块测试案例表如表 8 所示.

表 8 订单模块测试案例表

用例编号	测试模块	测试项	操作步骤描述	预期结果	实际结果	测试结论
Order-01	订单模块	购物车数据聚合	在不同设备登录同一账号, 查看购物车列表.	购物车列表正确显示已添加的商品.	跨设备数据一致, 价格同步更新.	通过
Order-02	订单模块	并发库存扣减	模拟用户提交订单, 购买数量等于当前 SKU 剩余库存.	订单创建成功, 对应 SKU 库存精确扣减为 0, 不出现负数(超卖).	生成订单号, 数据库库存更新为 0.	通过

(4) 物流模块测试案例表如表 9 所示.

表 9 物流模块测试案例表

用例编号	测试模块	测试项	操作步骤描述	预期结果	实际结果	测试结论
Logistics-01	物流模块	动态轨迹查询	在“我的订单”中点击“查看物流”。	触发物流轨迹生成算法，根据发收件地址渲染带有时间轴的运输节点树。	成功渲染时间轴，轨迹逻辑符合地理常识。	通过
Logistics-02	物流模块	物流地图轨迹查看	在物流轨迹页面点击查看地图轨迹	地图上清晰显示发货地、运输途经点及收货地位置，轨迹路线显示连贯无异常正常加载地图轨迹	路线展示完整连贯	通过

6.2 系统性能优化策略

电商系统大多存在读取数据多、写入数据少的特点，而且大促活动时还会面临高并发的压力，为了防止接口出现响应变慢甚至服务崩溃的情况，“安琦购”平台在设计阶段就从缓存结构、数据库连接池和搜索引擎三方面对系统性能做了整体优化，保证平台在高访问量下依然能稳定运行^[15]。

在缓存方面，商品详情页的访问量很高，直接查询 MySQL 数据库容易造成连接数被占满，因此系统使用 Redis，把热门商品的基本信息和 SKU 数据转换成 JSON 格式提前缓存起来。用户请求会先查找缓存，只有缓存中没有对应数据时才会查询数据库，大大减轻了数据库的读取压力。

在数据库连接管理上，系统选用 Alibaba Druid 作为 MySQL 的连接池，并根据实际的并发情况调整核心参数：初始连接数设置为 5，最大活跃连接数设置为 20。同时开启 testWhileIdle 和 validationQuery 功能来检测连接是否可用，避免数据库回收空闲连接后出现通信错误，提升了事务运行的稳定性。

除此之外，为了解决搜索速度慢的问题，系统用 Elasticsearch 代替了 MySQL 模糊查询，通过 IK 分词器为商品名称、品牌和介绍等内容建立倒排索引，把原本全表扫描的 O(N)复杂度降到接近 O(1)，实现了毫秒级的精准商品搜索。

7 总结

由于在整套项目开展的具体开发过程之中，一直在把电商领域里最为核心的那些买卖业务需求给紧紧围绕着的，并且是把这些个需求给当成了最核心的基础底子，去构建出来了一套包含了客户端接入层、网关路由层、微服务应用层以及底层数据基础设施在内的四层逻辑框架；一是在微服务开展开发的那个阶段里，把后端的整体功能模块给拆分成了包括用户、

商品、订单在内的多个相互独立的微服务，这中间是借助到了 Nacos 这种工具，才算是实打实地把关于服务注册以及配置管理的相关工作给真正地落实到了位；二是能依托着 Spring Cloud Gateway 并且搭配上 JWT 这种技术，才总算是把那个具备了统一鉴权功能的访问入口给成功地搭建了起来。

前端层面使用 Pinia 进行全局状态管理，同时封装 Axios 拦截器，统一处理接口鉴权和异常信息，再依托 uni-app 框架，实现了微信小程序与 H5 端的一体化开发。

在商品模块的设计思路中，确实建立了以 MySQL 数据库为主存储核心、将 Elasticsearch 灵活应用于辅助检索的读写分离架构，这一架构的搭建，使得商品信息的搜索查询真正达到了毫秒级水准；一方面通过编写专门的适配器函数，有效解决了 SKU 多规格数据适配过程中面临的棘手难题，另一方面，借助多组件的协同配合，系统在应对海量商品数据时的整体检索效率也实现了质的飞跃。

对于订单支付模块而言，确实有采用这种轻量化的数据模型，将购物车里的各类琐碎数据妥善管理起来，再配合乐观锁机制，便平稳高效地完成了分布式环境中常见的库存消减操作；一方面在与微信、支付宝等平台进行支付连通时，能够将具体配置参数与核心业务逻辑彻底分离，另一方面还会有多重校验规则全程管控，确保支付流程的整体安全性真正落到实处。

出于项目实际情况的综合考量，物流模块在现阶段还没能获取到相关资质，也就没法直接将第三方服务接口完成对接；基于这样的现实条件，本文已经通过自主设计的形式搭建出一套物流轨迹生成算法，能够用来把真实包裹的完整流转过程给模拟得更为清晰。

当整套系统的开发工作都已经全部完成过后，本文便又采用等价类划分、边界值分析等测试方式，对平台内部核心的业务流程开展过全方位的功能校验，能够看到所有测试用例都已经达到了预设的预期效果，一方面能看出本课题所构建的这套微服务电商平台，在实用性与架构扩展性上的表现确实较好，能够充分满足中小型电商企业的日常经营需求，另一方面其在架构设计与性能调优方面的思考路径，也可被同类型电商开发项目作为有价值的参考。

参考文献

- [1] 吴子珺. 电子商务系统分析与设计[M]. 第3版. 北京: 机械工业出版社, 2024.
- [2] 龙芳, 吴勇灵. 微信小程序购物系统的设计与实现[J]. 现代信息科技, 2023, 7(23): 13-16.
- [3] Zhu Y R. Contract Management System Based on SpringBoot and Vue[J]. Advances in Computer, Signals and Systems, 2024, 8(4): 55-60.
- [4] 陈冈. RxJS+Vue.js+Spring 响应式项目开发实战[M]. 北京: 清华大学出版社, 2024.
- [5] 贾志杰. Vue+SpringBoot 前后端分离开发实战(第2版)[M]. 北京: 清华大学出版社, 2025.
- [6] 陈恒. Spring Boot 从入门到实战(第2版)[M]. 北京: 清华大学出版社, 2024.
- [7] 朱元涛, 江冬勤, 黄毅. Spring Boot 项目开发实践[M]. 北京: 清华大学出版社, 2024.
- [8] 赵俊杰, 葛敬军, 朱文婷. 基于微信小程序的校园二手书交易平台的设计与实现[J]. 科技与创新, 2024(9): 7-11.
- [9] 李梦雯, 刘彬. 微信小程序性能优化策略与实践研究[J]. 移动信息(英文版), 2024, 46(6): 270-272.
- [10] 张齐勋, 吴一凡, 杨勇, 等. 微服务系统服务依赖发现技术综述[J]. 软件学报, 2024, 35(1): 118-135.
- [11] 梁自豪, 荆浩, 李雪源. 基于微服务的中小型 B2C 电商抢购系统的设计与实现[J]. 长江信息通信, 2024, 37(4): 115-117.
- [12] 田松涛, 段元梅. 基于 SpringBoot 的线上商城平台设计[J]. 电脑编程技巧与维护, 2024, 30(8): 58-60.
- [13] 孙旭. 基于 Spring Boot 框架与 B/S 架构的公务用车预约平台设计[J]. Science World Journal, 2024, 3(2): 45-50.
- [14] Morshchinina L, Zubareva A, Volosiuk A A. A Comparative Study of Web Frontend Reactivity(Vue3)[C]. Proceedings of SPIE, 2025, 13651: 136510R.
- [15] Di Meglio S, Starace L L L. Evaluating Performance and Resource Consumption of REST Frameworks and Execution Environments[J]. IEEE Access, 2024, 12: 161649-161669.

Design and Implementation of AnqiGou Platform Based on WeChat Mini Program

Zhao Ruqing

(School of Mathematics and Big Data, Dezhou University, Dezhou Shandong, 253023)

Abstract: In the current development of large enterprise-level e-commerce platforms, the microservice architecture has become an essential system design principle. It delivers substantial practical value in multiple aspects such as improving the system's high-concurrency processing efficiency and enhancing the flexibility of business expansion. On the one hand, an in-depth analysis based on the complete development case of the Anqigou WeChat mini-program enables readers to clearly understand the operational logic of the technical solution combining Spring Cloud Alibaba and uni-app within the full-link e-commerce system. On the other hand, following the entire development workflow, this paper elaborates thoroughly on how to apply the core concepts of the architecture to properly solve practical engineering challenges encountered in project development, including cross-platform adaptation, high-performance product retrieval, and distributed business governance.

Keywords: Microservice; WeChat Mini Program; Spring Cloud Alibaba; Elasticsearch

谢 辞

岁月总会在不经意间缓缓流转，随着本篇毕业论文已然顺利完成定稿，安琦购平台的整体开发工作也已经实现了完整落地，我数年的大学求学时光也即将迎来圆满的收尾；在论文顺利撰写完成的当下，我发自内心地想要向指导老师表达最真切的心意与满心的感恩之情。

一方面在项目前期开展整体构思规划以及微服务架构设计的初始阶段，各类陌生的技术选型曾让我陷入深度的迷茫状态，也难以梳理清楚后续的整体开发思路，有了指导老师耐心的引导与细致的讲解，才协助我理顺了整套完整的开发逻辑，也能够让我逐步解开心中所留存的各类疑惑，进而明确往后学习与项目开发的整体前行方向；另一方面在完成 Spring Cloud Alibaba 组件整合、开展分布式事务处理的重要阶段里，老师会陪同我逐一排查 Nacos 配置异常、网关鉴权失败等诸多技术故障，还能将标准化的工程实操方式与问题排查逻辑逐一传授到我的学习过程当中。

老师严谨认真的治学修养与踏实钻研的学术作风，会让我在系统研发、程序调试以及性能优化的整个过程里，切实夯实自身的编程技术功底，也能够从中积累下诸多珍贵的学习感悟与实践经历；校园毕业从来都不会是人生道路的终点，反倒会成为全新成长旅程的开端，我会带着大学阶段沉淀下来的实践经验与内心感悟，在往后的技术钻研道路上稳步前行，在此也再次向指导老师献上由衷的敬佩与真挚的谢意。

作者：赵汝庆

2026 年 4 月 30 日